

CometKV: Signature-Indexed KV Cache Offloading with Sampled-Tail Attention for Long-Context LLM Decoding (Technical Report)

Jiaxu Zhu¹, Haixin Wang², Ming Dong², Jiayu Yuan¹, Ziang Xu³, Yajuan Du², Bolong Zheng^{1*}

¹Huazhong University of Science and Technology

²Wuhan University of Technology

³Wuhan Britain-China School

{jiaxuzhu, jiayuyuan}@hust.edu.cn zblchris@gmail.com

{hxwang, mingdong, dyj}@whut.edu.cn xuziang1228@gmail.com

ABSTRACT

Long-context large language model inference is constrained by the capacity and bandwidth demands of the key-value (KV) cache. CPU offloading preserves contexts beyond GPU capacity, but turns the KV cache into a growing per-request data store spanning the CPU-GPU memory hierarchy. Each decode step must identify, transfer, and reuse a small query-dependent subset of host-resident records. Existing systems trade off retrieval granularity, index footprint, and construction cost, while retrieval alone cannot recover the attention mass discarded by top- K selection. We present CometKV, a CPU-GPU KV-cache offloading system that jointly addresses retrieval error and the truncation bias of top- K attention. It introduces a compact GPU-resident signature index with a novel asymmetric design that compresses stored keys and scores them against a full-precision query. It supports token-level retrieval, single-pass construction, and incremental insertion. CometKV further combines deterministic selection of high-scoring tokens with importance sampling of the remaining tail, recovering information that selection-only methods discard. A co-designed CPU-GPU KV store coordinates record layout, data movement, and GPU caching through the same retrieval signal. Experiments on LongBench and RULER show that CometKV matches full-attention accuracy with a 2% token budget and outperforms an exact-score top- K oracle. At a 96K context on a single commodity GPU, it accelerates decoding by up to 15.8× over the evaluated sparse offloading baselines.

PVLDB Reference Format:

Jiaxu Zhu, Haixin Wang, Ming Dong, Jiayu Yuan, Ziang Xu, Yajuan Du, and Bolong Zheng. CometKV: Signature-Indexed KV Cache Offloading with Sampled-Tail Attention for Long-Context LLM Decoding (Technical Report). PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/zhujx001/CometKV>.

*Corresponding author.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Transformer-based large language models (LLMs) increasingly serve workloads that read long contexts but generate relatively few tokens, such as document-grounded question answering, retrieval-augmented generation, code understanding, and multi-step reasoning [24, 41, 43]. Context windows now reach 128K tokens for LLaMA-3.1 and one million tokens for Gemini 1.5 [32, 40]. However, serving these contexts remains expensive. A primary bottleneck is the key-value (KV) cache, which stores the key and value vectors of every previous token at every transformer layer. Its size grows linearly with context length, batch size, number of layers, and number of KV heads. For example, the BF16 KV cache of LLaMA-3.1-8B occupies approximately 12 GB at a 96K-token context. Together with roughly 16 GB of BF16 model weights, it exceeds the 24 GB capacity of a single commodity GPU such as the RTX 4090. Moreover, every decode step must read the cached keys and values, making long-context decoding increasingly limited by memory capacity and bandwidth [22].

These characteristics motivate a data-system view of the KV cache. Rather than treating it only as temporary model state, we view it as a per-request data store that grows by one record per token and spans the GPU-CPU memory hierarchy. Each decode step issues a query for the records most relevant to the current token. From this perspective, efficient long-context decoding poses three storage-system questions: (i) *which compact index can remain on GPU while ranking CPU-resident records*, (ii) *how should KV records be laid out to support contiguous PCIe transfers*, and (iii) *which records should a GPU cache retain across decode steps*?

Existing approaches alleviate KV-cache pressure in three broad ways. Eviction and compression reduce the GPU footprint by permanently discarding information [5, 25, 28, 46, 54], while GPU-resident sparse attention reduces the amount of KV data accessed at each decoding step but still requires the entire KV cache to fit in GPU memory [39]. Offloading instead preserves the complete KV cache beyond GPU capacity, by GPU-CPU-disk scheduling [35] or moving a query-dependent subset from CPU memory during decoding [8, 45, 52]. The latter avoids irreversible information loss, but transforms a device-capacity problem into an online retrieval problem. At every decode step, the system must rank host-resident records for a changing query, move a small subset over PCIe, and exploit reuse across consecutive steps.

Making this retrieval path practical requires more than accurate similarity search. A practical offloading index should provide

(i) *token granularity*: allowing each KV record to be retrieved independently; (ii) *compact GPU residency*: keeping the complete retrieval index on device at only a small fraction of the KV-cache footprint; (iii) *per-token appendability*: so that each newly offloaded token can join without rebuilding the index, and (iv) *single-pass prefill construction*: avoiding offline training, iterative fitting, or a separate post-prefill build phase. Existing systems make different compromises across these requirements. ShadowKV [38] and In-LLM [45] retrieve position-based blocks. RetroInfer [8] retrieves entire clusters constructed through segmented spherical k -means. Its incremental updates avoid rebuilding the full index, but still recluster newly accumulated segments. PQCache [19, 52] similarly fits subspace centroids through iterative k -means during prefill. MagicPIG [9] places its large LSH tables in host memory, while ParisKV [31] retains substantial quantization metadata for every token. No existing offloading index simultaneously provides all four properties (as summarized in Table 1).

We present CometKV, a CPU-GPU KV-cache offloading system that combines a compact retrieval index with an attention estimator designed for low token budgets. Its design separates two sources of accuracy loss that are often conflated. *Retrieval error* occurs when an approximate compact index misranks relevant tokens, whereas *truncation bias* remains even when the exact top- K tokens are selected. The latter arises because top- K attention discards the softmax tail and renormalizes the retained weights. Improving the retrieval index can reduce the first loss, but cannot remove the second. CometKV therefore couples a signature index for accurate token ranking with a sampled-tail estimator that represents information discarded by top- K attention.

To reduce retrieval error, CometKV represents each offloaded key with a 16-byte signature that remains on GPU. Its key insight is an asymmetric retrieval design in which CometKV compresses the many stored keys while preserving the current query in full precision. Before encoding a key, CometKV subtracts a mean key vector computed from the prompt. This removes the common component shared by most keys, allowing the limited signature bits to capture the differences that distinguish one token from another. It uses 120 sign bits to represent direction and one byte to preserve magnitude. At each decode step, the current query remains in full precision and scores these signatures through a continuous asymmetric rule, producing a fine-grained ranking from a compact representation. The scan reads only 16 bytes per candidate instead of a 256-byte BF16 key. Its prompt-dependent statistics are computed in one pass during prefill, and a newly offloaded token joins by appending one signature, without clustering, refitting, or rebuilding the index.

To reduce truncation bias, CometKV combines deterministic selection with importance sampling rather than relying exclusively on either. It assigns most of the budget to a deterministic head containing the highest signature scores, while using a small sampled tail to represent the many weaker tokens that may still matter collectively. The sampling distribution reuses the ranking scores that CometKV has already computed, adding no second retrieval index. Sampled tokens are fetched from CPU memory, scored using their original BF16 keys, and corrected by their sampling probabilities before being merged with the deterministic head. In this way, the head preserves the strongest attention contributions, while the

sampled tail recovers information that top- K attention would otherwise discard. Unlike MagicPIG [9], which relies on CPU-resident LSH tables and CPU attention, CometKV obtains both components from one GPU-resident signature scan and executes the complete attention path on GPU.

Fine-grained selection is useful only if the selected records can be transferred and reused efficiently. CometKV therefore co-designs a CPU-GPU KV store for the access pattern created by signature retrieval. It stores each key-value pair contiguously in pinned host memory and uses a fused Unified Virtual Addressing (UVA) gather to place selected records directly into GPU attention buffers, so each record fetched from the host moves as one contiguous transfer without intermediate staging. A GPU token cache exploits reuse across consecutive decode steps, while its eviction policy reuses the signature scores already computed for retrieval and requires no separate importance tracker. Therefore, CometKV coordinates indexing, data movement, and caching through the same retrieval signal. The signature determines what to retrieve, the fused layout determines how to transfer it, and score-based caching determines what remains on GPU.

Our evaluation confirms that these designs improve both accuracy and efficiency. Across LongBench [4] and RULER [17], CometKV matches full-attention accuracy using only a 2% token budget and outperforms the exact-score TopK oracle at this budget. At a 96K context on a single RTX 4090, CometKV delivers up to 15.8 $\times$ faster decoding than the evaluated sparse offloading baselines and achieves the lowest end-to-end latency among them.

To summarize, this paper makes the following contributions:

- We formulate KV-cache offloading as a storage problem and derive four requirements for a practical index, covering token granularity, compact GPU residency, per-token appendability, and single-pass prefill construction.
- We design a 16-byte GPU-resident signature index that combines key centering with continuous asymmetric scoring for accurate, online token retrieval.
- We design a hybrid attention estimator that retains high-scoring tokens deterministically and samples the remaining tail using the signature scores as its proposal.
- We build an end-to-end CPU-GPU KV store that co-designs record layout, data movement, and score-based caching. Extensive experiments on LongBench and RULER show that CometKV matches full-attention accuracy at a 2% token budget while providing the lowest end-to-end latency among the evaluated sparse offloading systems.

The rest of the paper is organized as follows. Section 2 reviews the KV cache and compact retrieval codes. Section 3 derives the four index properties and measures why raw sign bits and pure top- K selection fall short. Section 4 presents the signature index, the hybrid estimator, and the CPU-GPU KV store. Section 5 evaluates accuracy and efficiency against five sparse baselines. Section 6 reviews related work, and Section 7 concludes.

2 BACKGROUND

2.1 LLM Inference and the KV Cache

Within each transformer layer, the self-attention module projects every token into query, key and value vectors [41]. For one query

head and its associated KV head, decode-time attention for the current query q_t over the L_t visible tokens is

$$o_t = \sum_{i=1}^{L_t} \frac{\exp(q_t^\top k_i / \sqrt{d_h})}{\sum_{j=1}^{L_t} \exp(q_t^\top k_j / \sqrt{d_h})} v_i, \quad (1)$$

where k_i and v_i are the cached key and value vectors of token i , and d_h is the per-head dimension (e.g., $d_h = 128$ for LLaMA-3.1-8B).

Inference consists of two phases. Prefill processes all prompt tokens in parallel and materializes their KV records, whereas the decode phase generates one token at a time and repeatedly accesses the cached history [1, 22, 51]. To avoid recomputing previous keys and values, the inference engine retains them in a per-layer KV cache. The prompt records are created during prefill, after which decoding appends one new record for every generated token.

Grouped-query attention (GQA) [2, 34] shares each KV head among multiple query heads. Let H_q and H_{kv} denote the numbers of query and KV heads, respectively. Each KV head serves a group of $G = H_q/H_{kv}$ query heads. This reduces the cache size by a factor of G relative to multi-head attention, but the cache still grows linearly with context length, batch size, and the number of layers. Caching eliminates repeated KV computation, but dense attention must still read every cached key and value at each decode step. Long-context decoding therefore faces both a capacity bottleneck from storing the cache and a bandwidth bottleneck from repeatedly accessing it [22, 33, 35].

Attention in LLMs is often concentrated on a small, query-dependent subset of tokens [28, 39, 54]. Retrieval-based offloading exploits this sparsity by keeping the full KV records in CPU memory and transferring only a selected subset to GPU at each decode step. The current query is used to rank the cached tokens, after which the selected keys and values are fetched over PCIe and processed by the attention kernel. This avoids requiring the complete KV cache to reside on GPU, but makes token retrieval and data transfer part of the decode critical path.

Selecting the right tokens and estimating the full-attention output are distinct problems. A retrieval method may accurately identify the tokens with the largest logits, yet attention computed only over those tokens renormalizes the softmax over a smaller set. Even exact top- K selection therefore discards the aggregate contribution of the unselected tail and can produce an output different from full attention [9]. Sparse attention must consequently control both errors introduced by approximate retrieval and errors caused by truncating the attention distribution.

2.2 Compact Codes and Asymmetric Scoring

Selecting similar vectors from a large collection without scanning the full-precision representations is a classical retrieval problem [6, 19]. Representative approaches include sign-based codes, which provide compact approximations of angular similarity, and quantization-based codes, which approximate inner products or distances. CometKV builds its index on the first family and scores it with a rule borrowed from the second.

Sign random projection. Sign random projection (SRP), also known as random-hyperplane LSH or SimHash [6], converts a real-valued vector into a binary signature. Each random hyperplane passes through the origin and divides the vector space into two

halves, and one bit records which half contains the vector. As an illustration, take two projection directions in the plane, measuring the signs of the horizontal and the vertical coordinate. The vector $u = (2, 1)$ then receives the signature 11, and so does $2u = (4, 2)$, since rescaling never moves a vector across a hyperplane through the origin. Rotating into the fourth quadrant, $v = (2, -1)$ receives 10 and differs from u in one bit, while the antipode $-u = (-2, -1)$ receives 00 and differs in both. Vectors pointing in similar directions therefore tend to fall in the same halves and produce similar signatures, irrespective of their lengths.

Formally, for a random direction $g \sim \mathcal{N}(0, I)$, SRP records the bit $1[g^\top x \geq 0]$. A random hyperplane separates two vectors x and x' with probability $\theta(x, x')/\pi$, where $\theta(x, x')$ is the angle between them. Across many independent projection directions, the fraction of differing bits therefore concentrates around this normalized angular distance. Conventional SRP encodes both stored vectors and queries in this way and ranks candidates by their Hamming distance. Because the projection directions are fixed independently of the indexed data, each newly arriving vector can be encoded without retraining or rebuilding the index.

Asymmetric scoring of stored codes. When a large collection is searched by one query at a time, compressing both sides of the comparison is unnecessary. Asymmetric scoring compresses the stored vectors, which dominate the memory footprint, while retaining the current query in full precision. Product quantization uses this principle to compare an uncompressed query against compact database codes, so approximation is introduced only on the stored side [19]. CometKV applies the same asymmetry to sign-based retrieval. It compresses the many cached keys into compact signatures while using the full-precision query to produce a more discriminative score than symmetric Hamming-distance comparison.

3 MOTIVATION

The preceding sections establish KV-cache offloading as a query-dependent retrieval problem. Although a small set of tokens often carries the largest individual attention weights, the remaining tokens may still contribute substantial aggregate mass. A practical offloading system must therefore retrieve strong candidates within the decode-time memory and latency budget while preserving attention contributions that token selection alone cannot capture. This section derives the corresponding system requirements and validates them through measurements of real LLM keys.

3.1 Requirements for Efficient KV Offloading

For analysis, we decompose the decode path into four logical components, some of which may overlap in an optimized implementation:

$$T_{\text{step}} \approx T_{\text{score}} + T_{\text{topK}} + T_{\text{PCIe}} + T_{\text{attn}}. \quad (2)$$

These terms represent candidate scoring, candidate selection together with any sampling, CPU-to-GPU transfer of the selected KV records, and attention computation, respectively.

The number of selected tokens is controlled by a budget. Let S_t denote the always-attended tokens at decode step t , including sink tokens at the beginning of the prompt and the most recent tokens [46]. These tokens remain on GPU and do not participate in retrieval. Given a budget ratio ρ , the number of entries retrieved

Table 1: Index properties of retrieval-based KV-cache offloading systems. Among the compared systems, only CometKV provides all four properties at once.

Method	Token granularity	Compact GPU residency	Per-token append	Single-pass build
InfiniGen [23]	✓	×	✓	×
ShadowKV [38]	×	×	×	×
MagicPIG [9]	✓	×	×	×
PQCache [52]	✓	×	✓	×
RetroInfer [8]	×	✓	×	×
ParisKV [31]	✓	×	✓	✓
CometKV	✓	✓	✓	✓

from CPU-resident tokens at step t is

$$K_t = \max(0, \lfloor \rho L_t \rfloor - |\mathcal{S}_t|). \quad (3)$$

We denote the retrieved tokens by $\mathcal{I}_t$ and the resulting attention set by $\mathcal{A}_t = \mathcal{S}_t \cup \mathcal{I}_t$.

Among the four terms in Eq. (2), exact scoring T_{score} is the only one that reads a d_h -dimensional full-precision representation for every candidate token. By comparison, T_{topK} accesses only one scalar score per candidate, while T_{PCIe} moves only the K_t selected records and T_{attn} runs over the attention set $\mathcal{A}_t$. At a 96K context it reads $L_t \times d_h$ BF16 values per KV head per layer, about 24.6 MB, and several gigabytes per decode step over 32 layers and 8 KV heads. Transferring a 2% subset, in contrast, moves approximately 1.0 MB of KV records per layer and KV head. Running the exact scan on CPU is constrained by host-memory bandwidth, while running it on GPU would first require transferring all candidate keys over PCIe. The central requirement is therefore to reduce T_{score} without sacrificing token-level selection.

This execution pattern imposes four requirements on the retrieval index.

- *Token granularity.* The index should score and retrieve each KV record independently. Block- or cluster-level retrieval may transfer many irrelevant records together with one relevant token, wasting both the token budget and PCIe bandwidth.
- *Compact GPU residency.* The complete index should fit in GPU memory at a few percent of the KV-cache footprint. Otherwise, candidate ranking would require traversing a host-resident index or repeatedly transferring index metadata over PCIe on the decode critical path.
- *Per-token appendability.* When a newly generated token is offloaded, it should become retrievable by appending one index record. The insertion should not rebuild the existing index, recluster accumulated tokens, or update context-sized fitted state.
- *Single-pass prefill construction.* All prompt-dependent index construction should complete within one model prefill, without offline training, iterative fitting, or a separate post-prefill build phase. Any fitted state should remain constant in size as the context grows.

Table 1 applies these requirements to existing offloading systems. ShadowKV [38] retrieves position-based KV blocks, while RetroInfer [8] admits or excludes entire similarity clusters. MagicPIG [9] supports token-level retrieval, but its LSH tables occupy 4.7–14 GB at 64K–96K contexts for LLaMA-3.1-8B and therefore reside in CPU memory. ParisKV [31] also selects individual tokens but retains centroid identifiers, direction codes, and scaling factors

for every token. PQCache [52] and RetroInfer construct their indexes through iterative clustering over prompt keys. These systems satisfy different subsets of the requirements, but none provides all four simultaneously.

The four index requirements primarily reduce T_{score} , but candidate scoring is only one part of the offloading path. Once tokens are selected, T_{PCIe} depends on how their KV records are organized and whether previously fetched records can be reused. Token-level retrieval may access records scattered across host memory, so each key-value pair should be transferable as one contiguous unit without intermediate staging. When consecutive decode steps select the same records, retaining them on GPU can further avoid redundant PCIe transfers. Efficient offloading must therefore co-design the retrieval index, host-memory layout, and GPU caching policy.

3.2 Feasibility and Limits of Compact Signature

The preceding requirements favor a compact per-token signature that remains on GPU and is scored in place of the full key. We next examine whether such a signature can preserve the ranking information needed for attention. Our measurements reveal two findings. Angular similarity retains most of the information in exact query-key scores, making compact angular codes promising, but conventional sign signatures provide insufficient discrimination at the low token budgets targeted by offloading.

Angles carry most of the ranking information. Attention ranks tokens by the inner product

$$q_t^\top k_i = \|q_t\| \|k_i\| \cos \theta(q_t, k_i).$$

The query norm is identical for every candidate, leaving the key norm and angular similarity to determine their ranking. We measure their relative importance using samples from five LongBench tasks across all layers and KV heads of four long-context models.

Table 2 shows that key norms are nearly uniform, with coefficients of variation (CV) between 0.043 and 0.094. This limited variation suggests that angular similarity, rather than key magnitude, carries most of the information needed to rank candidates. We verify this implication by comparing top-K selection based on cosine similarity with selection based on exact inner products. We define the attention mass of a selected set as the sum of its tokens’ softmax weights under full attention. At a 2% budget, cosine-based selection retains at least 97.3% of the mass captured by exact inner-product selection, and the attention outputs produced by the two selected sets have a cosine similarity of at least 0.991. These results indicate that key magnitude acts mainly as a secondary correction and that a compact angular code can, in principle, preserve the tokens important to attention. This observation is consistent with prior evidence that angular LSH sampling can preserve attention quality at system scale [9].

Sign random projection is also compatible with the four index requirements derived above. A fixed projection encodes each key independently into a 16-byte signature, allowing the complete index to remain on GPU and a newly offloaded key to be inserted by appending one record. The construction requires neither iterative fitting nor index rebuilding. The remaining question is whether such a compact signature provides sufficient angular resolution on real LLM keys.

Table 2: Cosine vs. inner-product top- K selection at a 2% budget. Cosine selection retains at least 97.3% of the attention mass of exact inner-product selection.

Model	Key Norm CV	Mass Ratio	Output Cos Sim
LLaMA-3.1-8B	0.089	0.997	0.999
Qwen2.5-7B	0.074	0.975	0.991
Mistral v0.2	0.094	0.998	0.997
GLM-4-9B	0.043	0.973	0.992

Table 3: Key geometry under a 128-bit sign signature on LLaMA-3.1-8B. Keys occupy a narrow cone around their prompt mean; keys sharing no direction would instead give a cosine of 0 and half their bits differing.

Measured quantity	Value
Cosine between a key and the prompt mean μ	0.70
Angle around μ holding 90% of the keys of a head	50°
Fraction of bits differing between two signatures	0.33

Raw sign signatures provide insufficient discrimination. To answer the question, we encode every post-RoPE key as $\text{sign}(Pk_i)$ under a fixed orthogonal projection $P \in \mathbb{R}^{128 \times 128}$, and measure the result on LLaMA-3.1-8B over CWE, MK3, Qasper, and 2WikiMQA with 32 samples each, at four layers and all eight KV heads. Two problems follow.

First, the keys do not spread uniformly across the vector space. As shown in Table 3, they concentrate around μ , the mean prompt key of their layer and KV head, with an average cosine similarity of 0.70 to μ and 90% of keys lying within 50° of it. Two keys that both lean toward this shared direction subtend an angle of about 60°, rather than the 90° of unrelated directions. Since a random hyperplane separates two vectors with probability θ/π , their signatures differ in about a third of their bits rather than half, matching the 0.33 measured in Table 3. The shared component thus consumes much of the signature capacity while contributing little to distinguishing individual tokens.

Second, a Hamming distance is an integer, so a 128-bit signature sorts candidates into only 129 groups and the set of tokens within a distance grows in large jumps. Over CWE and Qasper we measure randomly selected layers and KV heads. In one of them, 1.74% of the context lies within distance 65 of the query signature and 2.82% within distance 66. A 2% budget therefore takes every token within distance 65 and draws the remaining 0.26 percentage points from the 1.08 points that tie at distance 66, so about an eighth of the budget is filled by the newest-first tie-break rather than by similarity to the query. The tie bucket is intrinsic to an integer-valued distance at a low budget, not to this head.

These observations suggest three changes to conventional sign retrieval. First, centering keys around μ removes the shared direction without changing their exact inner-product ranking, since subtracting $q^\top \mu$ shifts every candidate score by the same constant. MagicPIG reports the same key concentration and also applies mean centering [9]. Second, retaining the query in full precision replaces integer Hamming buckets with continuous scores. Finally, while

raw key norms are nearly uniform, the centered norm $\|k_i - \mu\|$ measures the strength of each token’s deviation from the shared mean and should therefore be preserved as a secondary ranking signal. Section 4.2 presents how CometKV realizes these implications.

3.3 Limits of Selection-Only Attention

Improving the signature reduces retrieval error, but accurate ranking alone does not guarantee accurate attention. Even an oracle that selects the exact top- K tokens discards all remaining terms from the softmax numerator and denominator. Renormalization then increases the weights of the retained tokens whenever the discarded tail carries non-negligible mass. No fixed-budget selection-only method can recover contributions from tokens that it does not evaluate.

The size of this loss depends on the workload. At a 32K context and a 2% budget, an exact-score top- K oracle falls 18 points below full attention on the RULER frequent-words extraction task, where the answer aggregates many individually weak attention edges. Because the oracle has perfect top- K recall by construction, this loss cannot be repaired by a more accurate retrieval index. The decode path must instead estimate part of the contribution from the unselected tail.

Together, these findings establish the design requirements for CometKV. The retrieval path needs a compact, appendable GPU-resident index that preserves token-level discrimination, while the attention path must represent both the strongest tokens and the discarded tail. The surrounding KV store must move and cache selected records without introducing a second retrieval index. The next section describes how CometKV realizes these requirements.

4 COMETKV SYSTEM DESIGN

4.1 System Overview

Figure 1 shows how CometKV realizes the requirements derived above as a unified CPU-GPU data path. CometKV separates compact retrieval metadata from the original KV records. For each token in the retrieval zone, the original BF16 key and value reside in CPU pinned memory, while a 16-byte signature of the key remains on GPU. Sink tokens and recent tokens remain GPU-resident and are always included in attention. The signature is used only to determine which CPU-resident records to access and how to allocate the sparse-attention budget. Once selected or sampled, a token is fetched in its original precision, and its attention logit is computed from the original key rather than its signature.

During prefill, CometKV computes the keys and values of the prompt tokens as part of the normal model forward pass. It transfers the retrievable KV records to CPU pinned memory and constructs a signature for each offloaded key. For each layer and KV head, it accumulates a prompt key mean, freezes the statistics needed to encode centered keys, and generates one signature per offloaded token. KV offloading and signature construction are overlapped with model execution, so the index is completed within the same model prefill without offline training, iterative fitting, or a separate post-prefill build phase. The resulting prompt-dependent state has constant size per layer and KV head, while the index grows by one compact record per token.

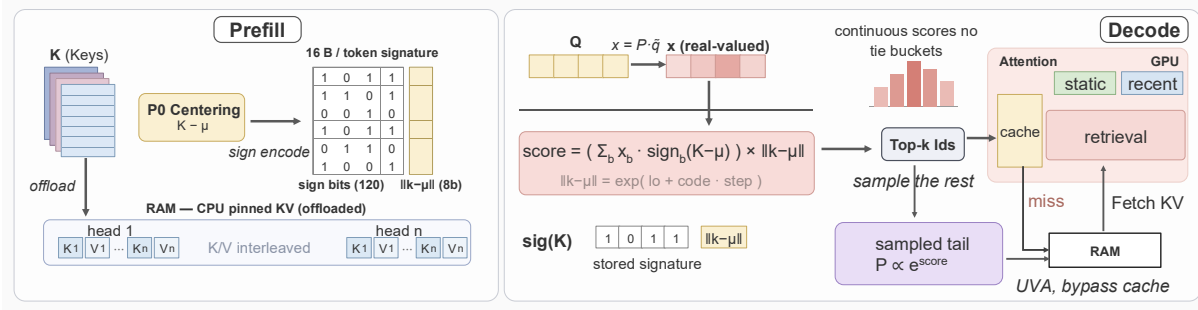


Figure 1: CometKV overview. Original BF16 KV records reside in CPU pinned memory, while a 16-byte signature per offloaded key remains on GPU. During decoding, one signature scan produces the deterministic head, the sampled-tail proposal, and the GPU-cache priorities. Original KV records are transferred only after retrieval and are used to compute the final attention output.

At each decode step and layer, CometKV projects the group-averaged query to a full-precision vector in the signature space. A GPU scan scores every retrievable signature against this full-precision vector and produces one score per candidate. These scores drive two query-dependent operations. CometKV selects the highest-scoring records to form a *deterministic head* and samples from the remaining candidates to form a *sampled tail*. The score vector also supplies the priorities used by the GPU token cache, so sampling and cache management need no index of their own. Together, token selection and tail sampling determine the KV records needed for the current attention computation.

After selection and sampling determine this working set, CometKV assembles the required KV records on GPU. The always-attended sink and recent-window tokens are already GPU-resident. Deterministic-head records that hit in the GPU token cache are served directly from device memory, while head misses and sampled-tail records are fetched from the token-contiguous CPU store. Attention is then computed from the original BF16 keys and values, with the sampled contributions corrected for their sampling probabilities before being combined with the deterministic component.

The following subsections present the compact asymmetric signature index, hybrid attention estimator, and CPU-GPU KV store in dataflow order.

4.2 Compact Asymmetric Signature Index

The first storage-system question posed in the introduction asks how a compact GPU-resident index can rank CPU-resident KV records. CometKV answers this question with a 16-byte signature for every offloaded key. Each signature contains 120 sign bits that encode the direction of the projected centered key and one byte that stores its log-quantized centered norm. The stored keys are therefore compressed into compact direction-and-magnitude records, while the query remains in full precision. This design targets retrieval error by removing the shared key component that reduces the discrimination of raw sign signatures and replacing coarse symmetric Hamming distance with a continuous asymmetric score. Unless stated otherwise, all index sizes and operations in this subsection are described per layer and KV head.

Centered projection and frozen statistics. CometKV constructs a projection matrix $P \in \mathbb{R}^{128 \times d_h}$ once by orthogonalizing a seeded Gaussian matrix. Orthogonal projection directions avoid spending

bits on correlated directions and sharpen the angle estimate for a fixed signature length [20]. The same matrix is shared across all layers and KV heads, so it introduces no per-layer or per-head projection state. The hyperplane-separation probability of Section 2.2 depends only on the angle between two vectors, so the per-layer, per-head adaptation lives in the frozen statistics below. For each layer and KV head, CometKV computes the mean of the prompt keys μ . Rather than projecting the raw key k_i , CometKV projects its centered form $k_i - \mu$. Centering removes the component shared by most prompt keys and allows the signature to represent the variation that distinguishes individual tokens.

Importantly, centering does not change the exact inner-product ranking. For any query q ,

$$\langle q, k_i - \mu \rangle = \langle q, k_i \rangle - \langle q, \mu \rangle. \quad (4)$$

The second term is identical for every candidate in the same layer and KV head. Subtracting it shifts all scores by the same constant without changing their order. Centering therefore improves the binary representation while preserving the ranking that the signature aims to approximate. The prompt mean and the statistics used for magnitude quantization are computed during prefill and then frozen. Their size is constant with respect to context length. This avoids iterative fitting and ensures that signature construction completes within the same model prefill.

Direction and magnitude encoding. For retrieval, a centered key contains two complementary sources of information. Its direction determines its angular similarity to the query, while its centered norm measures how strongly the key deviates from the prompt mean. CometKV encodes both within 16 bytes. The first 120 coordinates of the projected centered key are converted to sign bits and packed into 15 bytes, representing the direction of $k_i - \mu$. The final byte stores a log-quantized representation of its centered norm $\|k_i - \mu\|$. Preserving both components allows CometKV to distinguish keys that point in similar directions but deviate from the prompt mean by different magnitudes.

CometKV quantizes the centered norm in the logarithmic domain:

$$\text{code}_i = \text{clamp}\left(\text{round}\left(\frac{\log \|k_i - \mu\| - \text{lo}}{\text{step}}\right), 0, 255\right), \quad (5)$$

where lo is the minimum log centered norm among the prompt keys (i.e., $\text{lo} = \min_m \log \|k_m - \mu\|$) and step divides the prompt range into 255 equal intervals (i.e., $\text{step} = (\max_m \log \|k_m - \mu\| - \text{lo})/255$).

During scoring, CometKV reconstructs the centered norm from this code. The 256 available values cover the prompt range by construction, while a key generated during decoding is clamped to the nearest endpoint if its norm falls outside that range. At $d_h = 128$, a full BF16 key occupies 256 bytes, so the signature scan reads one-sixteenth as many bytes per candidate.

Continuous asymmetric scoring. Conventional SRP converts both the key and the query into bit strings and ranks candidates by integer Hamming distance. This symmetric comparison loses two forms of ranking resolution. First, a fixed-length signature produces only a small number of integer distance buckets, forcing many candidates to receive the same score. Second, Hamming distance treats every bit mismatch equally. A mismatch on a projection coordinate where the query has a large magnitude contributes exactly the same distance as a mismatch where the query is close to zero. Binarizing the query therefore discards information about which projection directions are most important to the current retrieval. CometKV avoids this loss by making scoring deliberately asymmetric. It compresses the many stored keys into binary signatures but retains the current query in full precision. The stored representation remains compact, while the score uses the magnitude of each query coordinate to weight the corresponding key bit.

Under GQA, several query heads share the same KV head. CometKV averages the query vectors in this group into one representative query $\tilde{q}$, allowing them to share one signature scan and one retrieved token set. It then projects this representative query as $x = P\tilde{q}$ and retains x in FP32. The representative query is used only for retrieval. After the selected KV records are fetched, each query head computes its own attention logits using its original query and the original BF16 keys. Specifically, CometKV scores signature i as

$$\hat{s}_i = \left(\sum_{j=1}^{120} x_j \cdot \text{sign}_j(k_i) \right) \cdot \exp(\text{lo} + \text{code}_i \cdot \text{step}). \quad (6)$$

Here, $\text{sign}_j(k_i) \in \{-1, +1\}$ is the stored sign of the j -th coordinate of $P(k_i - \mu)$. The sum compares the full-precision projected query with the binary direction of the centered key, while the exponential term reconstructs its centered magnitude. Their product approximates the centered query-key score used for ranking.

A simple example illustrates the difference from Hamming distance. Suppose two key signatures each disagree with the binarized query in exactly one bit. Hamming distance assigns them the same score, regardless of which bit differs. If one mismatch occurs on a query coordinate with magnitude 2.0 and the other on a coordinate with magnitude 0.1, CometKV penalizes the first mismatch much more strongly. It therefore distinguishes candidates that symmetric Hamming distance places in the same bucket.

Together, the full-precision query eliminates coarse Hamming buckets, while the norm byte restores centered-key magnitude, yielding a continuous, magnitude-aware ranking within a 16-byte per-token footprint. The key direction nevertheless remains a one-bit-per-projection approximation, leaving a residual ranking error in exchange for keeping the complete index on GPU.

Top- K selection, score reuse, and online insertion. At each decode step and layer, the signature scan computes one score in FP32 for every retrievable token and writes the results to a GPU buffer. A GPU top- K operator selects the highest-scoring records to form the

deterministic head. The operation ranks the computed signature scores exactly, although these scores remain approximations of the original query-key inner products. Section 4.3 describes how the total token budget determines the size of the deterministic head. The candidates outside the deterministic head form the sampling population for the tail. Their scores define the sampling proposal without requiring another index scan. The same score buffer also provides the priorities used by the GPU token cache in Section 4.4. CometKV therefore performs one signature scan and reuses its output for top- K selection, tail sampling, and cache replacement.

The index also grows online as decoding generates new tokens. Newly generated tokens first remain in the GPU-resident recent window and participate in every attention step. When a block of tokens leave this window, CometKV appends each token’s original key-value pair to the CPU store. Its signature is computed using the frozen prompt mean, projection matrix, and norm range, and then appended to the GPU index. The token becomes retrievable without rebuilding the index or refitting any prompt-dependent statistics.

Taken together, the design provides token-granular retrieval, a 16-byte GPU-resident index, per-token insertion, and construction within one model prefill. It assumes that prompt-derived statistics remain representative during generation; adapting them under substantially longer decode-time shifts remains future work.

4.3 Hybrid Attention Estimation

The signature index reduces retrieval error, but it cannot remove the truncation bias identified in Section 3.3. Even an exact-score top- K oracle discards every token outside the selected set and renormalizes softmax over the retained tokens. CometKV therefore estimates part of the discarded tail rather than relying on selection alone. It assigns most of the token budget to a deterministic head containing the strongest candidates and uses the remaining budget to sample weaker candidates whose aggregate contribution may still be important.

Deterministic head and sampled tail. CometKV splits the retrieval budget K_t of Eq. (3) into two parts. The deterministic head holds the $K_t - m_t$ candidates with the highest signature scores. Retaining these candidates deterministically keeps the highest-scoring retrieval results out of the random draw, which reduces estimator variance. The remaining m_t entries form a sampled tail. CometKV draws them with replacement from the retrievable tokens outside the deterministic head. Their proposal distribution is $p_j \propto \exp(\beta \hat{s}_j / \max(\sigma, 10^{-6}))$, where $\hat{s}_j$ is the signature score from Eq. (6). Because signature scores are computed in projection space rather than on the attention-logit scale, their numerical spread varies across layers and KV heads. Dividing by the standard deviation σ of the candidate scores removes this spread, and the scale β , with $\beta = 2$ by default, prevents the proposal from becoming either nearly uniform or excessively concentrated.

The proposal reuses the GPU score buffer already produced by the signature scan. CometKV only needs an additional pass to rescale these scores and draw from the result, requiring no second retrieval index. Accordingly, K_t counts attention entries rather than distinct token IDs. The deterministic head contributes $K_t - m_t$ distinct records, while the sampled tail contributes m_t draws whose

multiplicities are handled by the importance correction below. By default the tail takes a quarter of the budget, capped at 160 draws to bound its PCIe traffic. When the allocation falls below 64 draws, the head takes the full budget.

Importance correction. Sampling alone would overrepresent tokens with high proposal probabilities. CometKV corrects this imbalance when computing attention from the sampled tail. For every sampled occurrence of token j , it fetches the original BF16 key and value and computes the exact attention logit for each query head. The corrected logit is

$$\frac{q_t^\top k_j}{\sqrt{d_h}} - \log(m_t p_j), \quad (7)$$

where m_t is the number of tail draws and p_j is the probability of drawing token j . Subtracting $\log(m_t p_j)$ divides the token’s contribution by its expected number of occurrences, so frequently sampled tokens receive proportionally smaller weights and rarely sampled tokens receive larger ones. If a token is drawn multiple times, its corrected contribution is included once for each occurrence.

This correction recovers the discarded tail at the level of the unnormalized softmax terms. In particular, writing ℓ_j for the exact logit of token j , $\mathbb{E}[\sum_{j \in \text{draws}} e^{\ell_j - \log(m_t p_j)}] = \sum_{j \in \text{tail}} e^{\ell_j}$, and the same argument applies to the value-weighted numerator. CometKV therefore obtains unbiased estimates of both the tail numerator and denominator before the clipping and cross-layer reuse introduced below. Their ratio, like other self-normalized importance-sampling estimators, is not strictly unbiased, but it accounts for tail mass that deterministic top- K attention discards entirely. The deterministic head and the variance-control mechanisms described below keep this remaining estimation error small in practice.

Variance control. The signature-based proposal is not exact. Signature quantization changes the candidate ranking, and the group-averaged query used for retrieval does not exactly match every individual query head. A token may therefore receive a low proposal probability even when its exact per-head logit is large. The resulting correction can become excessively large, allowing one sampled entry to dominate the attention output.

CometKV limits this effect by clipping each corrected sampled logit at δ natural-log units above the mean corrected logit of the sampled tail, computed separately for each query head over its own sampled entries. The default value is $\delta = 4$. A larger value clips fewer entries but allows heavier importance weights, while a smaller value improves stability at the cost of greater clipping bias. Clipping therefore trades the expectation guarantee of the ideal estimator for more stable long-context generation.

Cross-layer sample reuse. Sampling a new tail at every layer would rebuild the proposal at each of them and issue each layer’s tail gather as a separate small transfer. CometKV instead redraws the tail every J layers and reuses the sampled token IDs and their proposal probabilities in the intermediate layers. The default is $J = 8$. Each intermediate layer still fetches its own KV records for the reused IDs and computes exact logits from its own original keys. Only the sampled token IDs and correction probabilities are reused.

This optimization amortizes tail sampling and allows the corresponding KV transfers to be scheduled across a small group of layers.

It also introduces an additional approximation. The expectation identity above holds directly at the layer that draws the samples. At an intermediate layer, the deterministic head is recomputed from that layer’s scores while the tail is reused from the earlier sampling layer. A reused tail token may therefore also enter the new deterministic head, or a token excluded by the earlier proposal may become important in the current layer. The reported results include this cross-layer reuse error; reducing J refreshes the proposal more frequently at a higher runtime cost.

Scope. The estimator targets fidelity to full attention. On tasks where full attention itself is not the accuracy ceiling and the answer hinges on ranking near-tied quantities, such as distinguishing words whose occurrence counts differ by little, the variance of a sampled tail can cost more than the truncation bias it removes. Section 5.2 reports such a case. For these workloads the sampled tail can be disabled by setting $m_t = 0$, reverting the decode step to pure selection with the full budget in the head. The experiments in Section 5 keep one fixed configuration and never disable the tail per workload, apart from the selection-only ablation arms of Section 5.4; an automatic switch driven by the spread of the candidate scores remains unevaluated.

4.4 CPU-GPU KV Store and Decode Execution

The preceding subsections determine which KV records participate in attention. The CPU-GPU KV store should then deliver these records without splitting any record across separate reads or staging it through intermediate buffers, since the selected token IDs are scattered across host memory. It should also exploit the different reuse patterns of the two estimator components. In particular, deterministic-head records are often selected again in subsequent decode steps, whereas sampled-tail records provide less predictable temporal reuse. CometKV addresses the two remaining storage questions through a token-contiguous CPU layout and a score-guided GPU cache.

Data placement and fused KV layout. CometKV keeps the n_s sink tokens on GPU and attends to them at every decode step, together with a recent window that starts as the last n_r prompt tokens; the remaining prompt tokens form the retrieval zone. The retrieval-zone tokens keep their original BF16 keys and values in CPU pinned memory, while their 16-byte signatures remain on GPU. Generated tokens enter the recent window. Once n_r new tokens accumulate, the window releases its oldest n_r tokens in one block, so the window holds between n_r and $2n_r$ tokens while Eq. (3) subtracts the configured $n_s + n_r$ at every step. The released tokens join the retrieval zone, their KV records in the CPU store and their signatures in the GPU index.

Within the CPU store of each layer, CometKV combines the batch and KV-head dimensions into a row index, $\text{row} = i_B \cdot H_{kv} + h_{kv}$, and places the key and value of each token next to each other:

$$\text{KV}_{\text{cpu}} \in \mathbb{R}^{(B \cdot H_{kv}) \times T_{\text{cpu}} \times 2 \times d_h}. \quad (8)$$

This layout gives the signature index, CPU store, GPU cache, and per-step attention buffer a common row-and-token address. More importantly, a cache miss requires one contiguous transfer containing both the key and value, corresponding to 512 bytes for BF16 records with $d_h = 128$. It also makes prompt offloading a sequence

of contiguous writes and allows newly offloaded records to be appended without reorganizing existing data. The CPU store and signature index are preallocated to the maximum request length T_{cpu} , so insertion writes one record and advances the valid-length counter.

CometKV constructs this layout during prefill. As each layer produces its prompt keys and values, their KV records are transferred into the corresponding CPU rows and the key statistics required by the signature index are accumulated. Once the statistics for a layer are available, CometKV encodes its prompt keys in bulk. The transfers and encoding kernels are pipelined with prefill execution across layers, avoiding a separate index-building phase after prefill.

Score-guided GPU token cache. Deterministic-head records exhibit substantial reuse across consecutive decode steps. On Qasper, between 63% and 75% of the tokens in the top 10% by exact attention score at one step remain in that set at the next step when aggregated across layers. CometKV exploits this locality with a GPU token cache that serves repeated selections from device memory and transfers only cache misses over PCIe.

Each row, corresponding to one request and one KV head, has an independent 8-way set-associative cache. A token ID maps to one set, and each way stores one BF16 key-value pair together with its token ID and an access stamp. Row-local set associativity bounds lookup and replacement work without requiring a global cache directory. The cache capacity of each row is configured as a small multiple of its deterministic-head size, with a default factor of two. This provides room for both the current working set and records that may be selected again in subsequent steps. For LLaMA-3.1-8B at a 96K context and a 2% retrieval budget, the cache occupies approximately 0.4 GB in total.

The replacement policy reuses the signature scores already computed for retrieval. Empty ways are filled first, and records selected in the current decode step are protected while the required working set is assembled. Among the remaining ways, CometKV evicts the resident record with the lowest current signature score, using the oldest access stamp to break ties. If every way in a set is currently protected, it falls back to least-recently-used replacement. The policy therefore favors records that remain relevant to the current query while requiring no separate importance predictor or context-sized priority structure.

End-to-end decode pipeline. After the signature scan produces the deterministic head and sampled tail, CometKV assembles their original KV records on GPU. A fused gather kernel copies deterministic-head cache hits into a contiguous GPU attention buffer. For cache misses, the same kernel reads the corresponding key-value pairs directly from CPU pinned memory through Unified Virtual Addressing (UVA), writes them into the attention buffer, and installs them in the selected cache ways. UVA does not eliminate PCIe traffic, but avoids intermediate staging buffers and additional host-initiated transfers. Sampled-tail records bypass the cache because their cross-step reuse is less predictable and caching them could displace frequently reused deterministic records. CometKV gathers these records for a cross-layer reuse window on a separate CUDA stream, allowing their PCIe transfers to overlap computation in earlier layers.

All attention logits are computed from the original BF16 keys rather than from their signatures. One FlashAttention call processes the always-attended sink and recent-window tokens together with the deterministic-head records. A fused kernel computes the sampled-tail logits, applies the importance correction and clipping described in Section 4.3, and combines the sampled and deterministic components using their log-sum-exp statistics. The resulting output uses a common softmax normalization over the always-attended tokens, deterministic head, and sampled tail.

Cost and memory accounting. The cost of one decode step decomposes term by term following Eq. (2), per layer and KV head. T_{score} falls from reading the 256-byte BF16 key of every candidate to reading its 16-byte signature, a $16\times$ cut that leaves about 1.5 MB of scan reads at a 96K context. T_{topK} operates on the materialized scalar scores rather than on the key vectors. T_{PCIe} carries one 512-byte KV pair per deterministic-head miss plus the cache-bypassed sampled tail, whose size is capped. T_{attn} runs over the attention set $\mathcal{A}_t$ instead of the full context. GPU state consists of the signature index, about 0.4 GB at 96K over the 32 layers and 8 KV heads of LLaMA-3.1-8B and under 2 GB even at a 256K context, the token cache and the per-step sparse buffer, both sized by the budget K_t , and the frozen statistics, about 131 KB in total at batch size 1.

5 EXPERIMENTS

5.1 Experimental Setup

Benchmarks and models. We evaluate accuracy on 16 English-language tasks from the real-world LongBench benchmark [4] and all 13 tasks from the synthetic RULER benchmark [17]. For RULER we evaluate 50 instances per task at 32K, 64K, and 96K. Efficiency is evaluated mainly on RULER frequent-words extraction (FWE). The primary model is LLaMA-3.1-8B-Instruct [40], and the cross-model study of Section 5.4 adds Qwen2.5-7B-Instruct [47] and Mistral-7B-Instruct-v0.2 [21], all three using grouped-query attention (GQA) [2]. The detailed experimental settings, per-task scores, and per-task ablation results are presented in Appendices A, B, and C.

Baselines. We compare CometKV (CKV) with full attention, an exact-score top- K oracle, and five retrieval-based KV-cache offloading systems under the same model precision and generation protocol and matched retrieval budgets.

- *Full attention (Full)* computes exact attention over the complete KV cache at every decode step and serves as the accuracy reference.
- *TopK* uses no signature and no sampled tail. Under the same S_t and budget K_t as the sparse methods, it selects per KV head the top- K_t tokens by brute force over exact inner products with the group-averaged query of Section 4.2 and renormalizes the softmax over them. This exact-score counterpart of the selection stage of CometKV upper-bounds signature-based selection but retains the truncation bias of the discarded tail, and it is not deployable.
- *ParisKV (PAR)* [31] prunes candidates through centroid collisions and reranks them using quantized inner-product estimates.
- *RetroInfer (RET)* [8] organizes KV records in a vector index and retrieves entire similarity clusters.

- *MagicPIG (MAG)* [9] samples candidates from locality-sensitive hashing tables.
- *PQCache (PQC)* [52] ranks tokens with product-quantization codes.
- *ShadowKV (SHA)* [38] retrieves position-based blocks using a low-rank key representation. The 2% budget leaves it far fewer blocks than its authors evaluate.

Budget and common settings. We report the retrieval budget as the ratio ρ of Eq. (3). For block- or cluster-level methods it is converted into a block or cluster count, so the sparse methods attend to a matched number of attention entries per step on average. MagicPIG derives its retrieved fraction from its hash-table configuration, so we use the settings its paper adopts at these costs, $K = 10$, $L = 150$ at 2% and $K = 8$, $L = 75$ at 5% [9]. All sparse methods share the always-attended set S_t of $n_s = 4$ sink tokens and an $n_r = 32$ recent-token window, so Eq. (3) subtracts 36 for every method. Newly generated tokens enter the recent window and join attention without counting against the budget, identically for every method. Unless stated otherwise, experiments use batch size $B = 1$ and BF16 model precision.

Implementation and configuration. CometKV is implemented on PyTorch 2.8 with CUDA 12.8. Each baseline runs its open-source implementation with the environment and hyperparameters its repository recommends, apart from ρ and S_t above. For CometKV, the projection matrix P uses a fixed random seed of 1234 and the default 16-byte signature of Section 4.2. The sampled tail receives one quarter of the retrieval budget, is capped at 160 draws, and is disabled when this allocation gives fewer than 64 draws. Corrected sampled logits use a clip of $\delta = 4$, the proposal scale is $\beta = 2$, samples are refreshed every $J = 8$ layers, and the sampler seed is 1234 unless stated otherwise. No value is tuned per model, benchmark, or context length, and the prefill MLP runs in 32K-token blocks.

Metrics and measurement protocol. Generation quality is scored with the official LongBench and RULER scripts, identical for every method. Efficiency is measured as time to first token (TTFT), time per output token (TPOT), end-to-end latency, and throughput in generated tokens per second. TTFT covers the whole prefill path, including method-specific index construction and KV placement, TPOT measures decode time over a fixed 128-token generation, and end-to-end latency covers prefill and that generation.

Hardware. Experiments run on a server with two NVIDIA RTX 4090 GPUs, each with 24 GB of memory connected over PCIe 4.0, on a host with 96 AMD EPYC 7K62 cores and 256 GB of RAM. Unless stated otherwise, every method runs on a single RTX 4090.

5.2 Accuracy Results

LongBench. Table 4 reports only benchmark averages, with bold marking the best sparse-method value in each row and Full and TopK kept as references. CometKV averages 49.49 and 49.46 over the 16 LongBench tasks at the 2% and 5% budgets, against 48.90 for full attention, and holds the highest sparse-method average at both. It is above the TopK oracle at 2% and 0.15 points below it at 5%. Retrieval can exclude context that contributes only attention noise, which can lift a sparse method slightly above full attention on some tasks. We read these margins, here and on RULER at 96K, as matching

Table 4: LongBench and RULER accuracy on LLaMA-3.1-8B-Instruct. Values are 16-task LongBench and 13-task RULER averages; RULER runs at the 2% budget. CometKV has the highest sparse-method average in every row.

	Full	TopK	CKV	PAR	RET	MAG	PQC	SHA
LongBench 2%	48.90	48.26	49.49	47.68	48.26	48.52	45.89	34.52
LongBench 5%	48.90	49.61	49.46	47.89	48.68	48.87	47.02	42.48
RULER 32K	91.37	90.35	90.77	87.31	86.61	88.18	59.26	47.90
RULER 64K	86.79	83.65	86.22	81.51	79.28	84.35	50.49	61.85
RULER 96K	82.86	79.65	83.40	77.53	77.04	79.57	36.79	58.44

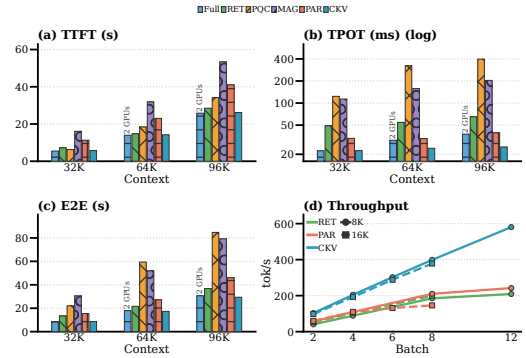


Figure 2: Efficiency on LLaMA-3.1-8B-Instruct at a 2% budget. CometKV has the lowest TTFT, TPOT, and end-to-end latency of the sparse methods; full attention uses two GPUs at 64K and 96K.

full-attention accuracy rather than a systematic advantage, because these runs do not separate them from benchmark variability.

RULER. On the 13 RULER tasks at the 2% budget, CometKV keeps the highest sparse-method average at every context length and exceeds the TopK oracle by a margin that grows with length. It stays within 0.60 points of full attention at 32K and 64K and averages 83.40 against 82.86 at 96K, and it leads the strongest sparse baselines, MagicPIG and ParisKV, at all three lengths. Section 5.4 traces these results to the signature index locating the relevant tokens and the sampled tail compensating for the truncation bias of selection.

5.3 Efficiency Results

The efficiency study runs FWE at 32K, 64K, and 96K with $\rho = 2\%$, and the batch-throughput panel at 8K and 16K at the same budget. Unless stated otherwise, numbers are means over five runs, with the spread far below the method gaps. Full attention fits on one RTX 4090 only at 32K and runs on two at 64K and 96K, split by layers. We exclude ShadowKV because its released CUDA kernels support only a fixed selection size.

TTFT. CometKV has the lowest TTFT among the sparse methods at every length (Figure 2(a)), 26.18 s at 96K against 28.50–53.31 s for the others. The baselines build codebooks, centroids, or hash tables over the prompt keys, whereas CometKV writes one 16-byte signature per token on GPU during prefill and stays within 0.35 s of full attention at every length.

TPOT. CometKV decodes at 22.43–25.17 ms per token from 32K to 96K, the lowest TPOT among the sparse methods at every length

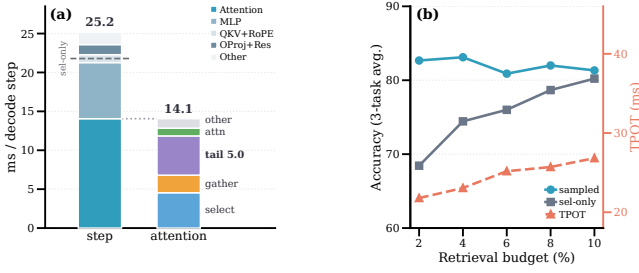


Figure 3: Decode-step breakdown and budget sweep at 96K. (a) Component shares on FWE at a 2% budget, scaled to the measured TPOT. (b) The sampled tail holds accuracy flat down to 2%; the TPOT curve excludes the tail.

(Figure 2(b)). At 96K, PQCache takes 15.8 $\times$ as long, scoring every candidate against product-quantization centroids each step, RetroInfer 2.6 $\times$ as long, fetching whole clusters, and MagicPIG reaches 203.88 ms. Reading 16 bytes per candidate and running the whole decode step on GPU keeps the CometKV TPOT nearly flat across the three lengths, whereas the full-attention TPOT rises from 22.31 ms at 32K to 37.68 ms at 96K even on two GPUs.

End-to-end latency. CometKV finishes the 96K workload in 29.38 s over prefix and 128 generated tokens (Figure 2(c)), the lowest in the study. RetroInfer, the fastest sparse baseline, takes 1.25 $\times$ as long. On one GPU CometKV finishes below the 30.62 s of two-GPU full attention, and at 32K, where full attention fits on one GPU, it trails by 0.28 s and still leads the sparse methods.

Throughput. At 8K and 16K, where larger batches still fit in 24 GB, CometKV scales best among the three measured methods (Figure 2(d)), reaching 1.9–2.8 $\times$ the throughput of RetroInfer and ParisKV at the largest batch of each length. We exclude MagicPIG and PQCache because their released implementations do not support batch sizes above one.

Time breakdown. Figure 3(a) decomposes one decode step of CometKV at 96K under the 2% budget. The component shares are five-seed medians from event-timed runs scaled onto the measured 25.17 ms steady-state TPOT, so the component times are estimates. Attention takes 14.1 ms of the step, of which the sampled tail costs 5.0 ms, the signature scan with top- K selection 4.5 ms, and the UVA gather 2.3 ms. Disabling the tail lowers the measured TPOT to 21.79 ms, a saving below the profiled 5.0 ms because the shares are estimates and the tail gather overlaps compute on a side stream.

5.4 Ablation Study

Effect of Scoring, Centering, and the Sampled Tail. Table 5 builds CometKV up component by component at a 2% budget, with the TopK oracle as the reference. Replacing the symmetric Hamming distance of Section 3.2 with the continuous asymmetric rule of Section 4.2, its norm byte included but the signatures still built from raw keys, raises the RULER average by 3.1–5.2 points. Continuous scoring and the norm byte together remove the integer bucket ties that decide selection at a low budget. Key centering adds a further 1.3–1.9 points, and the completed selection stack then matches the TopK oracle within 0.3 points at every length, so

Table 5: Ablation at a 2% budget; asterisks mark defaults. Selection reaches the TopK oracle and the sampled tail lifts the RULER average above it; the three-task average stays within 80.00–84.44 except at $\delta = 0$ and $\beta = 4$; the fused layout and the token cache set the decode speed.

Configuration	RULER			LongBench
	32K	64K	96K	16 tasks
Symmetric Hamming	83.30	79.56	74.86	49.11
+ Asymmetric scoring	88.52	82.65	78.19	49.41
+ Key centering	90.37	83.94	79.74	49.61
+ Sampled tail	90.77	86.22	83.40	49.49
TopK oracle	90.35	83.65	79.65	48.26

Accuracy	Stride J			Cap m_t		
	1	8*	32	64	96	160*
Accuracy	82.00	82.67	80.00	80.00	84.44	82.67

Accuracy	Clip δ		Scale β	
	0	4*	1	2*
Accuracy	72.44	82.67	82.00	82.67

KV store on FWE at 96K	TPOT (ms)	PCIe transfer (MB)
Staged copy, separate K/V	64.2	253.0
+ Fused layout, UVA gather	30.9	253.0
+ GPU token cache	23.4	28.8

signature-ranking error is no longer the main source of the remaining loss. What remains is the truncation bias of top- K attention itself. The sampled tail raises the RULER average by a further 0.40, 2.28, and 3.66 points, the gain growing with context length, and lifts CometKV above the TopK oracle at every length. The tail is the only change between these two CometKV rows, so the table attributes that increase to it. On LongBench the sampled tail is neutral, 49.49 against 49.61, because the 64-draw guard of Section 4.3 disables it at this budget on inputs shorter than about 14.5K tokens, so the two configurations coincide on many inputs.

Effect of Retrieval Budget. We sweep the budget from 2% to 10% at 96K on FWE, MK3, and QA1, and Figure 3(b) reports the three-task average for the full configuration and the selection-only variant. Without the tail, accuracy climbs from 68.44 at 2% to 80.22 at 10%. With the tail the curve stays between 80.89 and 83.11 across the whole range, and the 2% point scores 82.67, above the selection-only variant at 10%, stable over five seeds. The sampled tail therefore substitutes for budget on these tasks, its benefit concentrated at low budgets. TPOT rises from 21.79 ms at 2% to 26.83 ms at 10%, and this curve excludes the tail, which adds about 3.4 ms per step under its fixed 160-draw cap.

Effect of Estimator Parameters. The sampled tail fixes four constants in advance, the proposal scale β , the tail fraction with its 160-draw cap and 64-draw guard, the clip δ , and the redraw stride J . The middle panel of Table 5 varies one constant at a time around the default arm on FWE, MK3, and QA1 at 96K. Two settings account for the largest drops. Removing the clip costs 10.23 points, and setting $\beta = 4$ over-concentrates the proposal on the highest-scoring candidates and costs 5.11 points. Every other arm stays between 80.00 and 84.44, and J and the cap mainly trade decode time, with $J = 1$ adding 14.3% to the step time of the FWE workload below while lowering the average by 0.67 points and the 320-draw cap adding 5.4% with no gain. Even the 64-draw arms, the smallest tail the guard admits, score 80.00 against 68.44 for selection alone. The β arms form a separate batch.

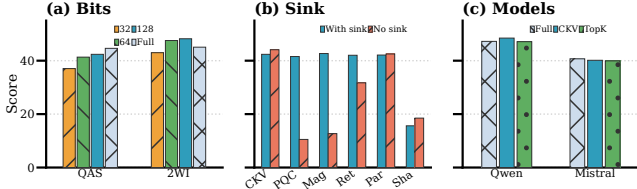


Figure 4: CometKV robustness. Accuracy improves with signature length (a), holds on Qasper without the sink and recent window (b), and matches or exceeds that of the TopK oracle on the 2% LongBench average for **Qwen2.5-7B and Mistral v0.2** (c).

Effect of the KV Store. The bottom panel of Table 5 isolates the KV store of Section 4.4 on FWE at 96K. These runs form a separate single-input batch, and transfer volumes count the fetched KV payload over all layers and KV heads, the two no-cache volumes coming from event-timed runs of the same configurations. The fused layout and UVA gather first halve the step time. Separate host K/V arrays with a staged contiguous copy move the same 253.0 MB per decode step, yet their gather takes 39.0 against 9.9 ms, because the fused UVA gather reads scattered 512-byte records at 25.6 GB/s, close to the practical PCIe 4.0 bandwidth, so its cost is set by the bytes moved rather than the access pattern. The GPU token cache then removes most of those bytes. It serves 96.6% of deterministic-head reads from device memory and cuts the per-step PCIe transfer to 28.8 MB, of which the cache-bypassing sampled tail contributes a fixed 21.0 MB. The generated output is identical across the three variants, and least-recently-used eviction changes the step time by less than 0.3 ms, so the gain comes from reuse on GPU rather than the eviction rule.

Effect of Signature Length. Figure 4(a) reports CometKV accuracy at signature lengths of 32, 64, and 128 bits on Qasper and 2WikiMQA. Every variant keeps the one-byte norm code, so the sign bits are 24, 56, and 120. These runs form a separate batch, sitting 1.5 points below the main accuracy batch on Qasper and 0.2 on 2WikiMQA at 128 bits. At 32 bits the score drops by 5.3 points on Qasper and 5.2 on 2WikiMQA, while 64 bits stays within 1.1 points of 128 bits.

Sink and Recent Window. On Qasper, removing the sink and recent tokens does not hurt CometKV, ParisKV, or ShadowKV (Figure 4(b)), while RetroInfer drops by 10.3 points and PQCach and MagicPIG by about 30, so signature retrieval locates the relevant tokens without the always-attended set.

Different Models. Figure 4(c) compares the 2% 16-task LongBench average of CometKV with full attention and the TopK oracle on Qwen2.5-7B and Mistral v0.2. CometKV reaches 48.46 on Qwen2.5-7B against the full-attention average of 47.23, a margin we again read as matching full attention, and 40.15 on Mistral v0.2, within 0.55 points of full attention. It matches or exceeds the TopK oracle on both, so one configuration holds accuracy on all three models.

6 RELATED WORK

Even with MQA and GQA [2, 34], the KV cache can exceed GPU capacity or saturate memory bandwidth [22, 52]. FlashAttention and FlashInfer reduce HBM traffic, PagedAttention reduces fragmentation, and Orca, Sarathi-Serve, and Apt-Serve improve batching and scheduling [1, 10, 11, 13, 22, 50, 51]. These techniques improve GPU-resident serving but cannot keep the complete KV cache on

GPU at long contexts. StreamingLLM, H2O, SnapKV, PyramidKV, and PyramidInfer retain only selected tokens [5, 25, 46, 48, 54], while KIVI, KVQuant, and KV-latent reduce the bytes stored per record [16, 29, 36]. Eviction cannot recover a token removed before the query arrives, while compression changes the stored representation used by attention. CometKV instead keeps model-precision KV records in CPU memory and compresses only the GPU-resident retrieval metadata.

Database indexes for approximate nearest-neighbor (ANN) search make related trade-offs in representation, updates, and placement. PQ Fast Scan places lookup tables in SIMD registers, DeltaPQ losslessly compresses PQ codes, and RaBitQ stores one bit per dimension with an error bound for distance estimation [3, 12, 42]. Learnable compression studies memory-bounded embeddings, LSH-APG maintains an LSH-based graph under insertions, and VStream adapts LSH partitions and storage to vector streams [14, 53, 55]. PASE integrates ANN into PostgreSQL, GENIE searches a GPU inverted index, CAGRA parallelizes graph construction and search on GPUs, and Widmoser et al. partition inverted lists across disaggregated memory [30, 44, 49, 56]. CometKV instead builds request-local metadata during prefill, appends one 16-byte record per offloaded token, layer, and KV head, and scans the signatures on GPU. Its signatures use sign random projection [6] and asymmetric scoring against a full-precision query, related to asymmetric distance computation and LSH [18, 19].

FlexGen schedules weights, activations, and KV across GPU, CPU, and disk, CacheGen streams compressed KV, and ArkVale reloads evicted entries [7, 27, 35]. InfLLM, InfiniGen, QUEST, SparQ, and OmniKV attend to query-dependent subsets [15, 23, 33, 39, 45], while RetrievalAttention, RetroInfer, PQCach, and HeteroCache fetch selected KV from CPU memory [8, 26, 37, 52]. ShadowKV reconstructs selected KV from a low-rank key cache, while ParisKV uses analytic centroids, 4-bit reranking, and online insertion [31, 38]. Unlike the clustering indexes of PQCach and RetroInfer, CometKV requires no fitted codebook or centroid; its token-contiguous CPU store and GPU token cache reuse the retrieval scores to select records and set cache priorities.

Selection alone drops the softmax tail and renormalizes the retained tokens. MagicPIG samples tokens through CPU-resident LSH tables and computes attention on CPU [9], while RetroInfer represents unretrieved clusters with centroids. CometKV drives its deterministic head and its tail-sampling proposal from one GPU signature scan. It fetches the sampled BF16 records and corrects them by the sampling probabilities, estimating the discarded tail without a second index.

7 CONCLUSION

CometKV keeps the offloaded KV cache in CPU pinned memory at model precision, scores 16-byte signatures on GPU against a full-precision query, and estimates attention with a deterministic head plus a sampled tail. The index meets the four properties of Section 3.1. At a 2% budget CometKV matches or exceeds the exact-score oracle on all three models, leads sparse methods in benchmark averages and latency, matches full attention on LongBench and at 96K on RULER, and at 96K finishes faster on one GPU than full attention on two; longer generations remain future work.

REFERENCES

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *OSDI*. USENIX Association, 117–134.
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *EMNLP*. Association for Computational Linguistics, 4895–4901.
- [3] Fabien André, Anne-Marie Kermaec, and Nicolas Le Scouarnec. 2015. Cache locality is not enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan. *Proc. VLDB Endow.* 9, 4 (2015), 288–299.
- [4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *ACL (1)*. Association for Computational Linguistics, 3119–3137.
- [5] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. 2024. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *CoRR* abs/2406.02069 (2024).
- [6] Moses Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *STOC*. ACM, 380–388.
- [7] Renze Chen, Zhuofeng Wang, BeiQuan Cao, Tong Wu, Size Zheng, Xiuhong Li, Xuechao Wei, Shengen Yan, Meng Li, and Yun Liang. 2024. ArkVale: Efficient Generative LLM Inference with Recalable Key-Value Eviction. In *NeurIPS*.
- [8] Yaoqi Chen, Jinkai Zhang, Baotong Lu, Qianxi Zhang, Chengruidong Zhang, Jing Liu, Jingjia Luo, Di Liu, Huiqiang Jiang, Qi Chen, Bailu Ding, Xiao Yan, Jiawei Jiang, Chen Chen, Mingxing Zhang, Cheng Li, Yuqing Yang, Fan Yang, and Mao Yang. 2026. RetrolInfer: A Vector Storage Engine for Scalable Long-Context LLM Inference. *Proc. VLDB Endow.* 19, 5 (2026), 1016–1031.
- [9] Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Léon Bottou, Zhihao Jia, and Beidi Chen. 2025. MagicPIG: LSH Sampling for Efficient LLM Generation. In *ICLR*. OpenReview.net.
- [10] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *ICLR*. OpenReview.net.
- [11] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*.
- [12] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 3 (2024), 167.
- [13] Shihong Gao, Xin Zhang, Yanyan Shen, and Lei Chen. 2025. Apt-Serve: Adaptive Request Scheduling on Hybrid Cache for Scalable LLM Inference Serving. *Proc. ACM Manag. Data* 3, 3 (2025), 130:1–130:28.
- [14] Shenghao Gong, HaoBo Sun, Ziquan Fang, Liu Liu, Lu Chen, and Yunjun Gao. 2025. VStream: A Distributed Streaming Vector Search System. *Proc. VLDB Endow.* 18, 6 (2025), 1593–1606. <https://doi.org/10.14778/3725688.3725692>
- [15] Jitai Hao, Yuke Zhu, Tian Wang, Jun Yu, Xin Xin, Bo Zheng, Zhaochun Ren, and Sheng Guo. 2025. OmniKV: Dynamic Context Selection for Efficient Long-Context LLMs. In *ICLR*. OpenReview.net.
- [16] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. In *NeurIPS*.
- [17] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What’s the Real Context Size of Your Long-Context Language Models? *CoRR* abs/2404.06654 (2024).
- [18] Qiang Huang, Guihong Ma, Jianlin Feng, Qiong Fang, and Anthony K. H. Tung. 2018. Accurate and Fast Asymmetric Locality-Sensitive Hashing Scheme for Maximum Inner Product Search. In *KDD*. ACM, 1561–1570.
- [19] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (2011), 117–128.
- [20] Jianqiu Ji, Jianmin Li, Shuicheng Yan, Bo Zhang, and Qi Tian. 2012. Super-Bit Locality-Sensitive Hashing. In *NIPS*. 108–116.
- [21] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *CoRR* abs/2310.06825 (2023).
- [22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*. ACM, 611–626.
- [23] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. In *OSDI*. USENIX Association, 155–172.
- [24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.
- [25] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. In *NeurIPS*.
- [26] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. 2024. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. *CoRR* abs/2409.10516 (2024).
- [27] Yuhua Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *SIGCOMM*. ACM, 38–56.
- [28] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. In *NeurIPS*.
- [29] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen (Henry) Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *ICML (Proceedings of Machine Learning Research)*, Vol. 235. PMLR / OpenReview.net, 32332–32344.
- [30] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. 2024. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs. In *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*. IEEE, 4236–4247. <https://doi.org/10.1109/ICDE60146.2024.00323>
- [31] Yanlin Qi, Xinhang Chen, Huiqiang Jiang, Qitong Wang, Botao Peng, and Themis Palpanas. 2026. ParisKV: Fast and Drift-Robust KV-Cache Retrieval for Long-Context LLMs. *CoRR* abs/2602.07721 (2026).
- [32] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy P. Lillicrap, Jean-Baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, Ioannis Antonoglou, Rohan Anil, Sebastian Borgeaud, Andrew M. Dai, Katie Millican, Ethan Dyer, Mia Glaese, Thibault Sottiaux, Benjamin Lee, Fabio Viola, Malcolm Reynolds, Yuanzhong Xu, James Molloy, Jilin Chen, Michael Isard, Paul Barham, Tom Hennigan, Ross McIlroy, Melvin Johnson, Johan Schalkwyk, Eli Collins, Eliza Rutherford, Erica Moreira, Kareem Ayoub, Megha Goel, Clemens Meyer, Gregory Thornton, Zhen Yang, Henryk Michalewski, Zaeheer Abbas, Nathan Schucher, Anshel Anand, Richard Ives, James Keeling, Karel Lenc, Salem Haykal, Siamak Shakeri, Pranav Shyam, Aakanksha Chowdhery, Roman Ring, Stephen Spencer, Eren Sezener, and et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *CoRR* abs/2403.05530 (2024).
- [33] Luka Ribar, Ivan Chelombiev, Luke Hudliss-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. 2024. SparQ Attention: Bandwidth-Efficient LLM Inference. In *ICML (Proceedings of Machine Learning Research)*, Vol. 235. PMLR / OpenReview.net, 42558–42583.
- [34] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. *CoRR* abs/1911.02150 (2019).
- [35] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *ICML (Proceedings of Machine Learning Research)*, Vol. 202. PMLR, 31094–31116.
- [36] Luohe Shi, Zuchao Li, Lefei Zhang, Baoyuan Qi, Guoming Liu, and Hai Zhao. 2025. KV-Latent: Dimensional-level KV Cache Reduction with Frequency-aware Rotary Positional Embedding. In *ACL (1)*. Association for Computational Linguistics, 1535–1550.
- [37] Zhiyuan Shi, Qibo Qiu, Feng Xue, Zhonglin Jiang, Li Yu, Jian Jiang, Xiaofei He, and Wenxiao Wang. 2026. HeteroCache: A Dynamic Retrieval Approach to Heterogeneous KV Cache Compression for Long-Context LLM Inference. In *ACL (1)*. Association for Computational Linguistics, 43172–43187.
- [38] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yujie Chi, and Beidi Chen. 2025. ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM Inference. In *ICML (Proceedings of Machine Learning Research)*, Vol. 267. PMLR / OpenReview.net.
- [39] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. QUEST: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In *ICML (Proceedings of Machine Learning Research)*, Vol. 235. PMLR / OpenReview.net, 47901–47911.
- [40] Llama Team. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024).
- [41] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All

- you Need. In *NIPS*. 5998–6008.
- [42] Runhui Wang and Dong Deng. 2020. DeltaPQ: Lossless Product Quantization Code Compression for High Dimensional Similarity Search. *Proc. VLDB Endow.* 13, 13 (2020), 3603–3616.
 - [43] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*.
 - [44] Manuel Widmoser, Daniel Kocher, and Nikolaus Augsten. 2024. Scalable Distributed Inverted List Indexes in Disaggregated Memory. *Proc. ACM Manag. Data* 2, 3 (2024), 171.
 - [45] Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. 2024. InLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. In *NeurIPS*.
 - [46] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *ICLR*. OpenReview.net.
 - [47] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 Technical Report. *CoRR* abs/2412.15115 (2024).
 - [48] Dongjie Yang, Xiaodong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. 2024. PyramidInfer: Pyramid KV Cache Compression for High-throughput LLM Inference. In *ACL (Findings)* (*Findings of ACL*), Vol. ACL 2024. Association for Computational Linguistics, 3258–3270.
 - [49] Wen Yang, Tao Li, Gai Fang, and Hong Wei. 2020. PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension. In *SIGMOD Conference*. ACM, 2241–2253.
 - [50] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *MLSys*. OpenReview.net/mlsys.org.
 - [51] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *OSDI*. USENIX Association, 521–538.
 - [52] Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. 2025. PQCache: Product Quantization-based KVCache for Long Context LLM Inference. *Proc. ACM Manag. Data* 3, 3 (2025), 201:1–201:30.
 - [53] Hailin Zhang, Penghao Zhao, Xupeng Miao, Yingxia Shao, Zirui Liu, Tong Yang, and Bin Cui. 2023. Experimental Analysis of Large-scale Learnable Vector Storage Compression. *Proc. VLDB Endow.* 17, 4 (2023), 808–822.
 - [54] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *NeurIPS*.
 - [55] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proc. VLDB Endow.* 16, 8 (2023), 1979–1991.
 - [56] Jingbo Zhou, Qi Guo, H. V. Jagadish, Lubos Krcál, Siyuan Liu, Wenhao Luan, Anthony K. H. Tung, Yueji Yang, and Yuxin Zheng. 2018. A Generic Inverted Index Framework for Similarity Search on the GPU. In *ICDE*. IEEE Computer Society, 893–904.

A DETAILED EXPERIMENTAL SETUP

This appendix supplements Section 5.1, which defines the benchmarks, models, hardware, software, metrics, and measurement protocol. To prevent duplicated settings from drifting, the main-paper setup takes precedence and the appendix records only additional method-specific details.

A.1 CometKV Configuration

Table 6 records the complete default configuration used by the reported CometKV runs. The same values are used across models, benchmarks, and context lengths unless an ablation explicitly changes one value.

Table 6: Default CometKV configuration.

Component	Setting
Selector	Continuous asymmetric scoring over centered keys with fused selection disabled
Signature	16 bytes per key with 120 centered sign bits and one log-quantized norm byte
Projection	Fixed projection matrix with seed 1234
Always-attended set	Four sink tokens and a 32-token recent window
Budget	Ratio ρ includes the always-attended set and is converted to an equivalent token count for block and cluster methods
Sampled-tail allocation	One quarter of the retrieval budget with a minimum active size of 64 and a cap of 160 draws
Proposal scale	$\beta = 2$, with candidate scores divided by $\max(\sigma, 10^{-6})$ for candidate-score standard deviation σ
Importance correction	Logit correction with clip $\delta = 4$
Refresh schedule	Samples refreshed every $J = 8$ layers with sampler seed 1234
Tail guard	Sampling disabled and the complete retrieval budget assigned to the deterministic head when the tail allocation is below 64
GPU token cache	Eight-way set-associative cache with score-guided replacement and capacity multiplier two
KV materialization	Original BF16 keys and values in CPU pinned memory with fused UVA gather for cache misses
Prefill	Projection MLP processed in blocks of 32K tokens

At 96K and a 2% budget, the total attention budget is 1966 entries. The four sink tokens and 32 recent tokens leave 1930 retrieved entries. The default 160-draw cap yields a deterministic head of 1770 entries and a sampled tail of 160 entries.

A.2 Baseline Configurations

The common model precision, always-attended set, and budget rules follow Section 5.1. The following paragraphs record repository-specific settings for the 2% runs. For the 5% LongBench runs, the sparse methods use the matched budget defined in Section 5.1, and MagicPIG uses $K = 8$ and $L = 75$. Table 6 gives the CometKV configuration.

ParisKV uses PolarANN at a 2% budget with sink=4, recent=32, nprobe=150, a cache unit of eight, 450 cache clusters, and an update interval of 32.

RetroInfer uses a cache ratio of 0.02 and an estimation budget of zero. It sets sink=4 and recent=32, uses a page size of eight and two pages per cluster, and keeps a 32-cluster buffer.

PQCache runs pq_search with two sub-vectors per head, six bits per sub-vector, topr=1, Euclidean distance, GQA, sink=4, and recent=32.

At the 2% budget, MagicPIG uses $K = 10$ and $L = 150$. It uses sink=4, recent=32, dense_layers=[], and chunk=32768 at both reported budgets.

ShadowKV runs shadowkv_cpu with rank 160, a chunk size of eight, sink=4, and recent=32. It is included in the accuracy comparison and excluded from the efficiency study because its released CUDA kernels support only a fixed selection size.

TopK uses no signature and no sampled tail. Under the same S_i and budget K_i as the sparse methods, it selects per KV head the top- K_i tokens by brute force over exact inner products with the group-averaged query of Section 4.2 and renormalizes the softmax over them. This exact-score counterpart of the selection stage of CometKV upper-bounds signature-based selection but retains the truncation bias of the discarded tail, and it is not deployable. The implementation explicitly sets force_sink=4 and force_recent=32. In the efficiency study, Full attention uses FlashAttention in BF16 with greedy decoding. The 64K and 96K runs use two GPUs.

MagicPIG and PQCache are omitted from the throughput panel because their released implementations do not support batch sizes above one. All other exclusions and multi-GPU conditions match Section 5.3.

B PER-TASK ACCURACY RESULTS

The five tables in this section report every per-task score behind the benchmark averages in Section 5.2. They use the protocol of Section 5.1, and each Avg row can be recomputed from its task scores within two-decimal rounding. The unrounded mean for ShadowKV on LongBench at the 5% budget is 42.485, which is printed as 42.48 under the paper convention.

Bold marks the best value in each row among the six sparse methods CKV, PAR, RET, MAG, PQC, and SHA, and ties are all bolded. Full and TopK are reference columns and are never bolded. Every displayed value is reported to two decimal places. The exact method and task abbreviations used in the protected result tables are listed below.

Abbreviation	Method
Full	full attention
TopK	exact-score selection oracle
CKV	CometKV
PAR	ParisKV
RET	RetroInfer
MAG	MagicPIG
PQC	PQCache
SHA	ShadowKV

LongBench		RULER	
Code	Task	Code	Task
NQA	NarrativeQA	S1	single-key needle 1
QAS	Qasper	S2	single-key needle 2
MFQ	MultiFieldQA-en	S3	single-key needle 3
HOT	HotpotQA	MK1	multi-key needle 1
2WI	2WikiMultihopQA	MK2	multi-key needle 2
MUS	MuSiQue	MK3	multi-key needle 3
GOV	GovReport	VAL	multi-value needle
QMS	QMSum	QRY	multi-query needle
MNE	MultiNews	VT	variable tracking
TRE	TREC	CWE	common words extraction
TRI	TriviaQA	FWE	frequent words extraction
SAM	SAMSum	QA1	question answering 1
PCO	PassageCount	QA2	question answering 2
PRE	PassageRetrieval-en		
LCC	LCC		
RBP	RepoBench-P		

Table 8: LongBench per-task scores on LLaMA-3.1-8B-Instruct at the 5% budget. CometKV keeps the highest sparse-method average at 49.46, with MagicPIG second at 48.87.

Dataset	Full	TopK	CKV	PAR	RET	MAG	PQC	SHA
NQA	29.82	29.96	29.74	29.11	29.80	29.66	30.46	29.12
QAS	44.62	45.03	44.16	42.74	43.30	44.60	42.78	28.69
MFQ	55.55	56.57	56.11	54.68	56.12	54.15	49.80	49.10
HOT	55.65	57.75	57.57	57.19	56.51	55.83	54.07	55.03
2WI	45.06	47.39	47.29	46.62	45.57	44.34	42.35	45.28
MUS	30.33	32.78	32.43	31.79	30.02	30.74	29.27	29.36
GOV	35.07	34.50	34.33	34.27	34.59	34.46	32.67	23.21
QMS	25.19	25.82	25.52	25.03	25.29	24.87	25.01	22.79
MNE	27.34	25.22	26.08	25.96	25.92	26.94	21.46	13.98
TRE	72.50	72.50	72.50	71.50	72.50	72.50	68.50	65.00
TRI	91.17	90.96	91.92	91.12	91.29	92.12	91.74	81.41
SAM	43.58	44.06	43.50	42.74	43.42	42.81	42.19	40.95
PCO	7.28	10.77	10.00	8.50	7.28	7.78	6.22	6.22
PRE	99.50	99.50	100.00	99.50	99.00	99.50	99.50	98.50
LCC	63.43	63.33	63.31	54.60	61.00	63.61	58.42	44.21
RBP	56.35	57.64	56.86	50.94	57.30	58.05	57.86	46.91
Avg	48.90	49.61	49.46	47.89	48.68	48.87	47.02	42.48

B.1 LongBench Per-Task Results

Table 7 reports the 2% budget and Table 8 reports the 5% budget. Full attention has no retrieval budget, so its column is identical in the two tables.

Table 7: LongBench per-task scores on LLaMA-3.1-8B-Instruct at the 2% budget. CometKV has the highest sparse-method average at 49.49 and the best sparse-method score on 12 of the 16 tasks.

Dataset	Full	TopK	CKV	PAR	RET	MAG	PQC	SHA
NQA	29.82	29.74	30.65	28.79	30.43	29.06	30.10	28.59
QAS	44.62	43.98	43.90	42.11	42.03	42.68	41.56	15.62
MFQ	55.55	55.35	56.93	54.98	56.63	53.54	51.24	39.60
HOT	55.65	57.82	57.28	56.78	56.01	55.72	54.51	53.37
2WI	45.06	47.26	48.40	46.74	45.06	45.54	44.62	37.90
MUS	30.33	32.40	33.63	31.12	29.90	31.11	27.76	27.70
GOV	35.07	34.05	34.30	33.79	34.07	33.96	27.07	17.32
QMS	25.19	25.63	25.29	24.99	24.74	25.17	24.24	22.19
MNE	27.34	16.39	25.86	25.53	25.38	26.76	18.98	5.82
TRE	72.50	72.50	72.50	71.50	72.00	72.00	67.00	48.00
TRI	91.17	91.15	91.52	91.15	91.30	91.29	91.07	57.26
SAM	43.58	43.26	43.19	43.21	43.10	43.07	40.81	26.87
PCO	7.28	10.50	10.00	8.33	7.25	7.25	6.12	6.16
PRE	99.50	99.50	100.00	99.50	99.50	99.50	98.50	92.50
LCC	63.43	55.29	62.19	53.72	59.00	62.39	55.16	34.62
RBP	56.35	57.38	56.21	50.57	55.23	57.23	55.52	38.85
Avg	48.90	48.26	49.49	47.68	48.26	48.52	45.89	34.52

B.2 RULER Per-Task Results

Tables 9, 10, and 11 report the 2% budget at 32K, 64K, and 96K. The method and task sets are unchanged across the three lengths.

Table 9: RULER per-task scores on LLaMA-3.1-8B-Instruct at 32K under the 2% budget. CometKV has the highest sparse-method average at 90.77, and the aggregation tasks CWE and FWE separate the methods most.

Task	Full	TopK	CKV	PAR	RET	MAG	PQC	SHA
S1	100.00	100.00	100.00	100.00	100.00	100.00	42.00	90.00
S2	100.00	100.00	100.00	100.00	100.00	98.00	80.00	98.00
S3	100.00	100.00	100.00	100.00	100.00	100.00	62.00	2.00
MK1	100.00	100.00	100.00	100.00	100.00	100.00	96.00	84.00
MK2	100.00	100.00	100.00	100.00	96.00	100.00	84.00	48.00
MK3	100.00	100.00	98.00	98.00	88.00	96.00	16.00	0.00
VAL	100.00	96.50	100.00	89.00	97.50	99.50	48.50	48.00
QRY	100.00	100.00	100.00	100.00	100.00	99.00	63.50	43.00
VT	99.60	97.20	99.20	96.80	97.20	97.20	78.40	54.80
CWE	53.60	60.20	46.20	29.20	39.20	30.60	4.60	2.20
FWE	94.67	76.67	96.67	76.00	68.00	94.00	69.33	70.68
QA1	80.00	78.00	80.00	82.00	76.00	70.00	68.00	42.00
QA2	60.00	66.00	60.00	64.00	64.00	62.00	58.00	40.00
Avg	91.37	90.35	90.77	87.31	86.61	88.18	59.26	47.90

Table 10: RULER per-task scores on LLaMA-3.1-8B-Instruct at 64K under the 2% budget. CometKV has the highest sparse-method average at 86.22, ahead of MagicPIG at 84.35.

Task	Full	TopK	CKV	PAR	RET	MAG	PQC	SHA
S1	100.00	100.00	100.00	100.00	100.00	100.00	28.00	100.00
S2	100.00	100.00	100.00	100.00	98.00	100.00	88.00	100.00
S3	100.00	100.00	100.00	100.00	100.00	100.00	54.00	68.00
MK1	100.00	100.00	100.00	100.00	100.00	100.00	80.00	96.00
MK2	100.00	100.00	100.00	100.00	94.00	98.00	66.00	56.00
MK3	94.00	86.00	94.00	68.00	46.00	72.00	10.00	0.00
VAL	98.00	97.50	97.50	88.00	98.00	97.50	34.50	57.50
QRY	100.00	99.50	100.00	96.00	99.50	100.00	54.50	53.00
VT	100.00	91.60	99.60	93.20	93.20	98.40	48.80	78.80
CWE	13.00	14.20	9.80	2.40	4.00	8.00	1.20	0.60
FWE	91.33	70.67	90.00	80.00	72.00	92.67	75.33	78.14
QA1	72.00	72.00	74.00	74.00	72.00	70.00	64.00	64.00
QA2	60.00	56.00	56.00	58.00	54.00	60.00	52.00	52.00
Avg	86.79	83.65	86.22	81.51	79.28	84.35	50.49	61.85

Table 11: RULER per-task scores on LLaMA-3.1-8B-Instruct at 96K under the 2% budget. CometKV has the highest sparse-method average at 83.40, ahead of MagicPIG at 79.57.

Task	Full	TopK	CKV	PAR	RET	MAG	PQC	SHA
S1	100.00	100.00	100.00	100.00	100.00	100.00	10.00	100.00
S2	100.00	100.00	100.00	94.00	100.00	96.00	62.00	100.00
S3	100.00	100.00	100.00	100.00	100.00	98.00	24.00	68.00
MK1	100.00	100.00	100.00	100.00	100.00	98.00	58.00	100.00
MK2	98.00	98.00	98.00	98.00	90.00	90.00	50.00	40.00
MK3	84.00	58.00	84.00	28.00	36.00	66.00	4.00	2.00
VAL	97.50	97.50	97.50	90.50	93.00	93.00	19.50	42.50
QRY	100.00	100.00	100.00	95.50	99.50	98.50	40.00	62.50
VT	91.20	89.60	91.60	90.40	85.60	90.40	36.80	63.20
CWE	1.20	0.40	2.40	0.20	1.40	1.20	0.00	0.00
FWE	81.33	68.00	86.67	77.33	70.00	81.33	72.00	69.48
QA1	76.00	76.00	76.00	80.00	74.00	72.00	62.00	64.00
QA2	48.00	48.00	48.00	54.00	52.00	50.00	40.00	48.00
Avg	82.86	79.65	83.40	77.53	77.04	79.57	36.79	58.44

Table 12: Per-task RULER scores for the 96K budget sweep. Selection-only disables the sampled tail. Sampled uses the default quarter-budget tail with the 64-draw guard and 160-draw cap.

Configuration	Budget	MK3	FWE	QA1	Avg
Selection-only	2%	52.00	75.33	78.00	68.44
Selection-only	4%	70.00	77.33	76.00	74.44
Selection-only	6%	72.00	80.00	76.00	76.00
Selection-only	8%	80.00	80.00	76.00	78.67
Selection-only	10%	82.00	82.67	76.00	80.22
Sampled	2%	84.00	88.00	76.00	82.67
Sampled	4%	86.00	87.33	76.00	83.11
Sampled	6%	82.00	84.67	76.00	80.89
Sampled	8%	84.00	86.00	76.00	82.00
Sampled	10%	84.00	84.00	76.00	81.33

Table 13 expands the estimator-parameter panel into its MK3, FWE, and QA1 components. Each group varies one parameter around the default arm. The scale rows vary β while retaining the score normalization by $\max(\sigma, 10^{-6})$. Those rows form a separate batch, with the $\beta = 2$ reference taken from the canonical parameter grid.

Table 13: Per-task RULER scores for the estimator-parameter ablation at 96K and a 2% budget. A star marks the default value within each parameter group.

Parameter	Value	MK3	FWE	QA1	Avg
Refresh stride J	1	82.00	88.00	76.00	82.00
Refresh stride J	8*	84.00	88.00	76.00	82.67
Refresh stride J	32	84.00	78.00	78.00	80.00
Tail cap m_t	64	80.00	82.00	78.00	80.00
Tail cap m_t	96	86.00	91.33	76.00	84.44
Tail cap m_t	160*	84.00	88.00	76.00	82.67
Tail cap m_t	320	86.00	86.00	76.00	82.67
Clip δ	0	62.00	81.33	74.00	72.44
Clip δ	4*	84.00	88.00	76.00	82.67
Scale β	1	80.00	90.00	76.00	82.00
Scale β	2*	84.00	88.00	76.00	82.67
Scale β	4	74.00	82.67	76.00	77.56

The default 2% sampled arm in both tables is the same canonical configuration with a 160-draw cap, tail fraction 0.25, refresh stride eight, clip four, and proposal scale $\beta = 2$. The reported means are arithmetic means of MK3, FWE, and QA1 rounded to two decimal places.

C DETAILED ABLATION RESULTS

The budget and estimator-parameter ablations in Section 5.4 use LLaMA-3.1-8B-Instruct at 96K on MK3, FWE, and QA1. Each task contributes 50 instances. Table 12 expands the budget sweep into the three task scores behind every plotted average.